TECHNICAL OVERVIEW · OCTOBER 2026

Two AI systems built on home PCs: a self-improving context harness and a video understanding pipeline

How an Obsidian ontology graph gives an AI assistant shared, long-term memory, and how FFmpeg, Whisper, local vision models and Claude turn long videos into study notes and short clips.

2–7%
of the tokens Claude would need to read an 18:50 film frame by frame
measured

0.4–15%
of a mature project's notes loaded by a new session
measured

≈37%
overall token use vs the earlier chat-based workflow
team estimate

300–400 s
to grasp a 15-minute video from the outputs
team estimate

Source code and templates: the accompanying GitHub repository (README.md). Hardware: 16 GB RAM, 6–8 GB NVIDIA GPU, Windows 11. Only Claude is a paid subscription; every other model and tool is free and runs locally.

1. Summary

Large language models are strong at reasoning but have two practical limits: they forget everything between sessions, and they cannot watch video. This work addresses both with plain files and local tools rather than paid infrastructure.

System	Problem	Approach	Result
Context harness	The assistant forgets between sessions; re-reading old chats is slow, costly and tied to one machine.	An Obsidian ontology graph of small notes, read in a fixed short order; a lesson loop that turns mistakes into rules; Git sync across PCs and accounts.	New session loads 0.4–15% of a project's notes; ≈37% overall token use `est.` ; supported MUN training for 20+ delegates `est.`
Video pipeline	Understanding a long video means watching all of it; sending every frame to a model is far too expensive.	yt-dlp → FFmpeg → Whisper, Demucs, PySceneDetect, OpenCV, local Qwen3-VL → one compact "film card" that Claude reads.	18,000 tokens instead of 255,000–791,000 `measured` ; 300–400 s to grasp a 15-min video `est.`

2. System 1: the context harness

2.1 The ontology graph

Knowledge is stored as many short Markdown notes in Obsidian. Each note declares its parent in its front matter (`up: "[[INDEX]]"`) and links related notes in the text (`[[Note]]`). Parent links form a tree from one index; cross links form the rest of the graph. A central **Harness** branch holds the rules that apply to every project; each project holds a short **STATE** note and an append-only **run-log**. Some vaults are generated by code: `ontology.py` defines domains and concepts, and `build_vault.py` writes one linked note per concept and attaches real audience questions to it.

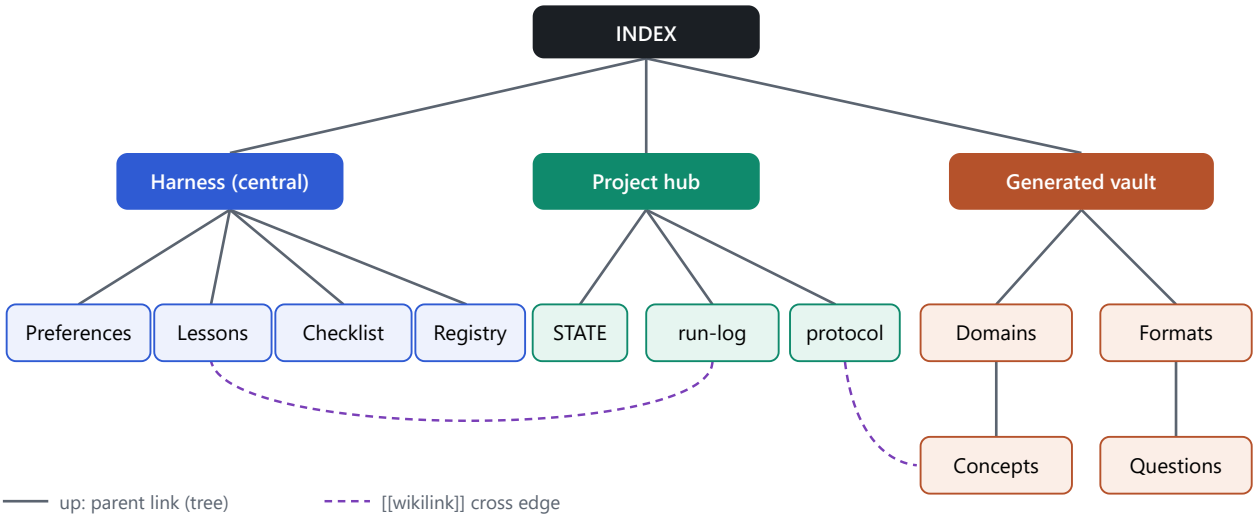


Figure 1. The vault as an ontology: parent links make a tree from one index; cross links connect a project's protocol to the lessons it came from, or a concept to the questions about it.

2.2 Context management: a fixed, short reading path

A new session never re-reads an old chat. It reads five things in order and then starts work. Lessons are opened only when something fails. Bulk data follows the same rule: local tools turn it into compact files first, and the assistant reads only those (Section 3).

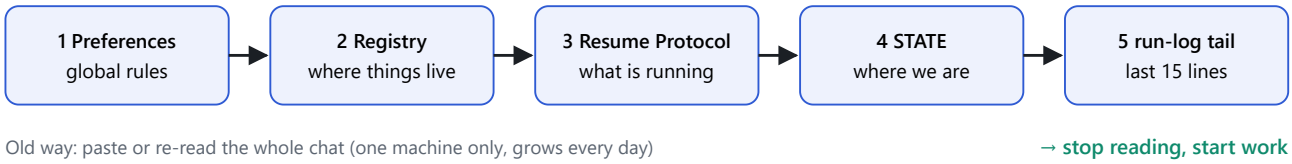


Figure 2. The resume path. Each note is short and current because the work loop rewrites STATE after every step.

2.3 Relearning: mistakes become rules

Every failure or correction is written as one row: **symptom, cause, fix**. When the same lesson appears a second time it is **promoted**: into a script that makes the mistake impossible, a line in the done-checklist, or a standing rule. The checklist runs before any task is called done.

New kinds of work run first with the strongest model and the user, and the exact protocol (every measurement, its script, thresholds, what an agent may and may not read) is written during those rounds. Cheaper models then run the job from the protocol alone. A model log records which model did best on which task type.

Example: a local vision tagger was right on only 40% of 50 hand-checked shots. The lesson (test on a gold set covering every label, define each label in the prompt) raised it to 76% and became a checklist rule.

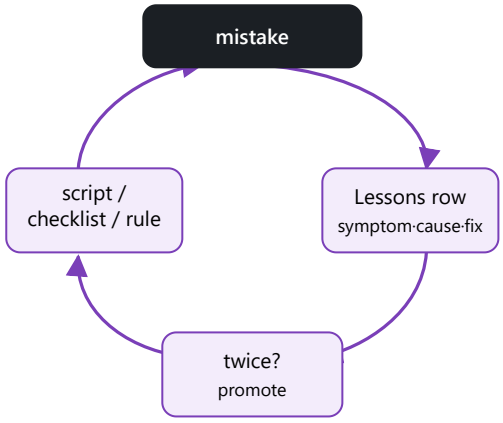


Figure 3. The relearning loop.

2.4 One context across PCs and accounts

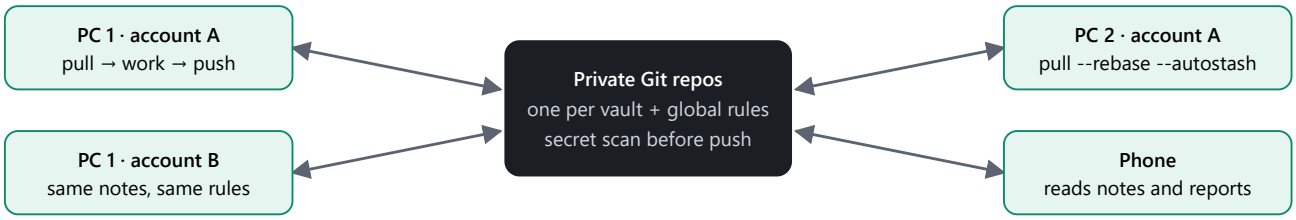


Figure 4. Every machine and assistant account starts from the same notes. Only changed notes are committed (never per-device editor files), each commit carries the device name, and a conflict stops the sync instead of overwriting work.

2.5 Measured effect

Project vault	Notes a new session loads	All notes in the project	Share
MUN Academy (about 1,000 notes)	3.0k tokens	719k tokens	0.4%
Web scraping	5.5k	126k	4%
Documentary	4.3k	48.1k	9%
Clipping research	7.7k	52.8k	15%
Assistant (young project)	5.8k	11.2k	52%

Measured 29 October 2026 on the real vaults; tokens estimated as bytes ÷ 4. The share falls as a project grows, because the resume path stays the same length while the notes keep growing. The ≈37% overall figure on the cover is the team's estimate for mixed daily work, which includes reading done during tasks, not only at resume.

3. System 2: the video understanding pipeline

A video is treated as audio and frames on one clock. Local tools split them, measure each, and merge the results by timestamp. The language model never watches the video; it reads a one-page summary, the transcript and a few contact sheets.

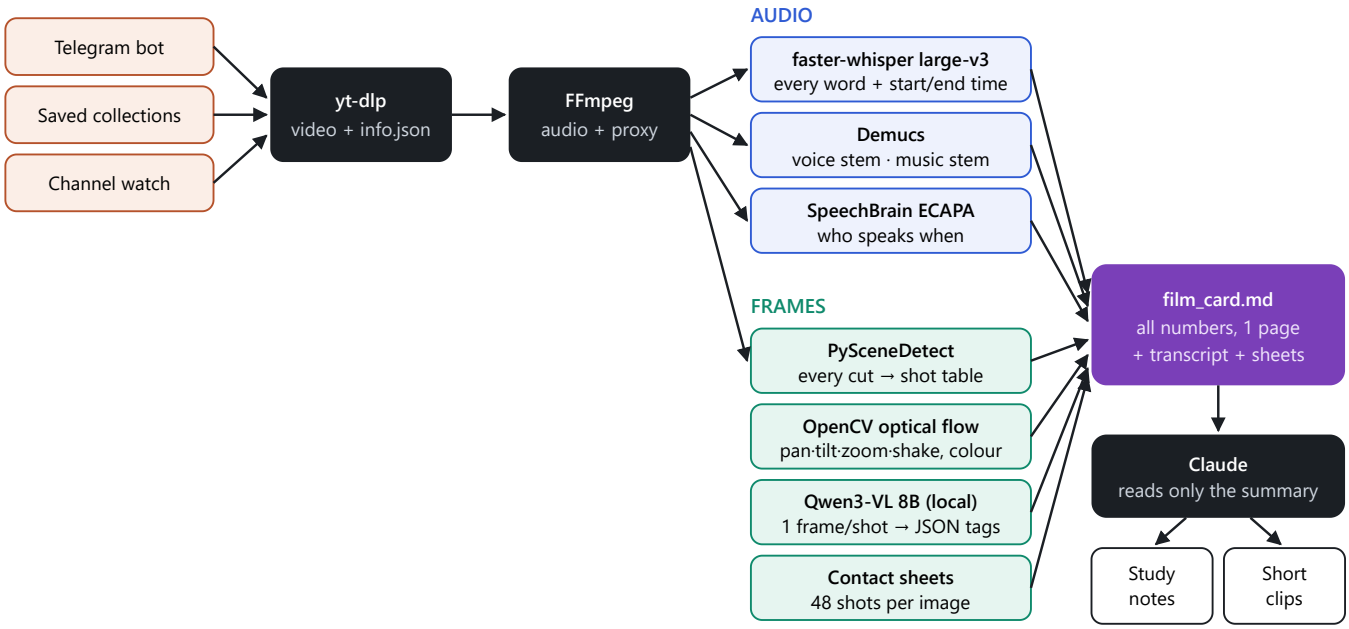
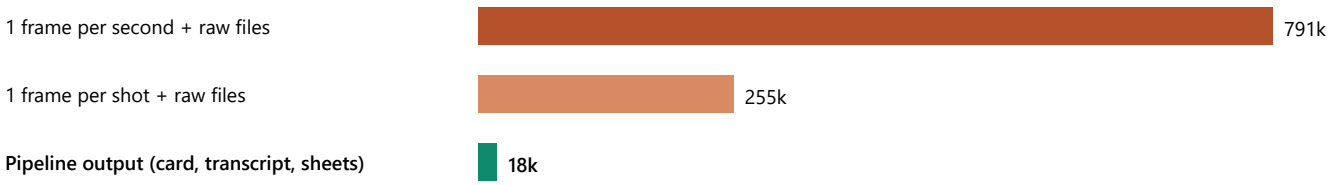


Figure 5. Pipeline architecture. Everything left of the summary card runs locally on the PC; only the card, transcript and sheets reach Claude.

3.1 Stages, tools and outputs

Stage	Tool / model	What it produces
Intake	Telegram bot (`reel_bot.py`), collection reader, channel watch	A clean link from Instagram, YouTube or TikTok, or a video file under 20 MB; only the paired owner can use the bot
Download	yt-dlp, with Node.js for YouTube's script challenge	Best video ≤1080p + best audio merged; info.json with title, views, chapters, "most replayed" heatmap, top comments
Split	FFmpeg (GPU encoder for the proxy)	16 kHz mono audio for speech, 44 kHz stereo for music, 960 px proxy video
Speech	faster-whisper large-v3 (GPU, CPU fallback)	Transcript with a start and end time for every word
Sound	Demucs htdemucs; SciPy signal analysis	Words per minute, pauses (and the line before each long pause), pitch, voice-over-music level, music change points, share of cuts placed on a sound hit vs chance
Speakers	SpeechBrain ECAPA + clustering	Voice clusters with share, pitch, pace, and a "phone line" signature
Shots	PySceneDetect; OpenCV	Every cut; per-frame camera motion; per-shot brightness, saturation and 5-colour palette
Visual tags	Qwen3-VL 8B via Ollama, JSON, temperature 0	Per shot: medium (3D, archival, document, map…), shot size, angle, subject, lighting, on-screen text, effects
Summary	`film_card.py`	One Markdown page with every measurement, by chapter
Understanding	Claude	Study document: structure, hook lines, techniques, timestamps

3.2 Measured token saving



Tokens Claude must read to understand an 18:50 documentary (256 shots). Images ≈ width×height/750 tokens; text ≈ bytes/4.

Figure 6. The model reads 2–7% of what a frame-by-frame approach would cost, with no information thrown away: the numbers cover every frame.

3.3 From a long stream to short clips

1. **Find moments.** Windows of 25–62 seconds that start on a new sentence and end on a finished one are scored on loudness bursts, sustained loudness, reaction words, an opening hook, a payoff at the end, speech density, the YouTube "most replayed" heatmap, and penalties for dead air and filler. Picks are spread at least three minutes apart; static menus are pushed down by an on-screen motion check.
2. **Judge.** Claude Haiku scores the best 25 in one call. If the nightly budget is spent, a local Qwen3 4B model judges instead.
3. **Cut.** FFmpeg builds 1080×1920 clips (facecam on top, gameplay below, or a blurred background), burns in word-timed captions, and normalises loudness.

4. **Write and check.** Claude Sonnet writes a label, three hooks, a title and a caption using what the team's study of 1,000+ clips found. Every text passes rule checks (length, blocked topics, quotes that must appear in the clip) and a local-model voice check.
5. **Human review.** Nothing is posted automatically; every clip waits for approval in a review note.

3.4 Responsible collection

> The downloaders behave like one careful person: the user's own logged-in session (cookies exported from their browser, never a typed password), a home connection rather than a server, at least 30 s between Instagram downloads, a 10-minute pause on any rate-limit warning, and a full stop with a STOP file on a login challenge or block. Comments are stored as text and like counts only, without usernames, and local videos are deleted after analysis. Certificate problems are fixed with the operating system's certificate store, never by turning checks off.

4. Engineering choices

Choice	Why
Two GPU stages (`cpu` , then `gpu`)	A 6–8 GB GPU fits one heavy model at a time: Whisper, then Demucs, then the vision model.
Resume-safe scripts	Every bulk job appends results and skips finished items, so a crash or reboot loses nothing.
Resource gates	Heavy steps wait for free RAM, VRAM and disk; a daily cap limits Claude calls.
Local first, Claude last	Free local models do the volume; the paid model does judgement and writing.
Status words	Every result is labelled *measured* (a script over all the data), *read* (a person or model looked), or *guess*.

5. Limitations

- The 37% overall token figure, the 300–400 s comprehension time and the 20+ delegates are the team's own estimates, not a controlled study.
- Camera-motion thresholds are not yet validated against hand labels; the vision tagger reached 76% on a 50-shot gold set.
- The language model reads transcripts and measured audio; it does not hear tone of voice.
- Scripts carry the authors' machine paths and need editing to run elsewhere.

6. Repository contents

Folder	Files
`context-harness/`	README, note templates, ontology generator (`ontology.py`, `build_vault.py`), `sync-vaults.sh`
`video-pipeline/1-intake/`	`reel_bot.py` (Telegram), `collect_links.py`, `loop.py`
`video-pipeline/2-film-analysis/`	`run_film.ps1`, `dl.py`, `transcribe.py`, `shots.py`, `sheets.py`, `audio.py`, `voices.py`, `tag.py`, `film_card.py`
`video-pipeline/3-shorts-clipper/`	`chief/` (queue, budget, gates), `moments/finder.py`, `cutter/cutter.py`, `writer/writer.py`, `review/`, `watch/`
`video-pipeline/4-clip-study/`	`cycle.py`, `build_training.py`, `measure_hooks.py`, `transcribe_reads.py`
`video-pipeline/example-output/`	`film_card.md` (18:50 documentary), `reel_timeline.md` (30 s reel)